实验 2_seq2seq 模型

提交人：GaoT

日期：2026 年 4 月

提交文件目录结构：

```
├── hw2_实验报告.pdf                    # 实验报告
├── seq2seq-translation_base.ipynb      # 基础 RNN 模型代码
├── seq2seq-translation_gru. ipynb      # GRU 模型代码
├── seq2seq-translation_attention. ipynb  # Attention+批处理代码
├── test_translations_batch.txt         # 训练集原文和模型推理译文
```

一、 基础模型(seq2seq_base)训练和评价

1、模型概况

参考代码提供了一个简单的 seq2seq 模型的实现，模型编码器和解码器都使用 RNN，模型训练采用迭代方法，每次迭代从训练集中随机抽取 1 个句子对进行训练。

本实验在原代码的基础上，做了以下几点修改：

（1） 特殊字符处理

<SOS>和<EOS>索引分别从 0、1 改成 1、2，0 保留供以后批量模型 PAD 用；增加<UNK>索引，方便处理验证集/测试集 OOV。

```
'''
考虑了验证集/测试集未登录词(OOV)的处理，在此表中加入了未知词索引，并设置低频词过滤
对原代码做了相应改动
'''
SOS_token = 1  # 修改原索引 0 (batch计算中0被用于padding)
EOS_token = 2  # 修改原索引 1 (1被用于SOS索引)
UNK_token = 3  # 添加未知索引

class Lang:
    def __init__(self, name, min_count=1):
        self.name = name
        self.min_count = min_count
        # 特殊词索引 (PAD=0 保留，但不一定使用)
        self.word2index = {"<SOS>": SOS_token, "<EOS>": EOS_token, "<UNK>": UNK_token}
        self.word2count = {}
        self.index2word = {SOS_token: "<SOS>", EOS_token: "<EOS>", UNK_token: "<UNK>"}
        self.n_words = 4  # 下一个新词索引从4开始
```

（2） 划分验证集

在数据预处理阶段从原来的训练数据中划分出 10%作为验证集，以便后续计算 BLEU。

数据集和词汇表概况：

```
Reading lines...
Read 90000 sentence pairs
Trimmed to 68898 sentence pairs
总数据集大小: 68898
训练集大小: 62009 (90.0%)
验证集大小: 6889 (10.0%)
Counting words on train data...
Building vocabularies on train data(min count=2)...
Input vocabulary size: 3883
Output vocabulary size: 8855
['她買了兩雙襪子。', 'she bought two pairs of socks .']
```

2、模型训练和评价

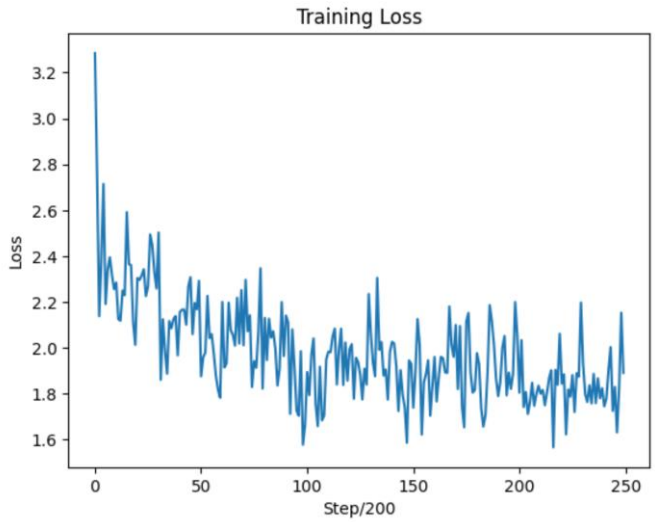
训练参数：

```
hidden_size = 500
n_layers = 2
dropout_p = 0.05
```

```
learning_rate = 0.0001
```

```
n_epochs = 50000  # 实际上是迭代次数
```

训练损失曲线：纵坐标为训练集损失，横坐标 1 个单位为 200 次迭代。由于每次迭代都是单句子对训练，损失值波动较大。经过 50000 次迭代，训练集损失已经下降到 2 以下，可看作基本收敛。



随机句子对推理结果：随机选择训练集和验证集中的句子进行推理比较，发现模型只能正确翻译出前几个简单单词，且会在句末生成大量句号[.]。

```
====== 随机训练集 ======
> 我打算買支筆送給他。
= i ll buy a pen for him .
< i m t to . . . .


====== 随机验证集 ======
> 他們在下棋。
= they re playing chess .
< they re not . .
```

验证集 BLEU：分别计算整个测试集的 BLEU 和句子的平均 BLEU。

```
Corpus BLEU: 0.1519
Avarage Sentence BLEU: 0.1411
```

BLEU 值与参考随机句子对的翻译结果一致，能翻译出一些常用词，但完全不能正确理解句子语义。

3、模型调优

3-1. 推理调优（Beam Search）

基础模型能翻译出高频单词，但会很快生成多个句号，我考虑到贪婪搜索的局限性，在不重新训练模型的基础上，采用束搜索 beam search 尝试优化推理算法。针对基础模型的翻译结果出现重复字符/单词，且句子长度较参考翻译短的问题，在束搜索算法中，还加入了 2 个约束：

（1） 重复惩罚（防止重复输出）

（2） 长度惩罚（为鼓励生成长度适中的句子，可以在排序时对较短序列增加惩罚）

3-2. 训练调优（超参数）

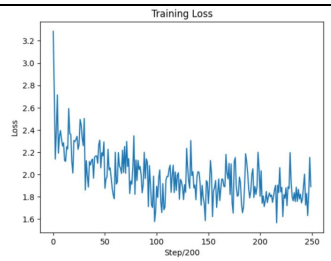
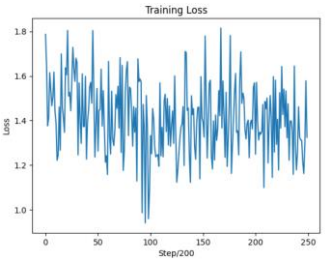
当前超参数分析：

超参数	当前值	评价
`hidden_size`	500	对于 6.9 万句对、词汇表 8k 的任务是合适的。可以保持不变。
`n_layers`	2	适中，能捕捉一定层次信息。若句子长度 ≤10，2 层足够。
`dropout_p`	0.05	偏低。如果训练集损失持续下降但验证集 BLEU 停滞或下降，可增大到 0.2~0.3。
`learning_rate`	0.0001	偏低，导致收敛慢，且可能陷入局部最优。建议提高到 **0.001** 或 **0.0005**。

优化后超参数：

```
# ---------- 超参数 ----------
hidden_size = 500
n_layers = 2
dropout_p = 0.2  # 提高（base：0.05）
learning_rate = 0.0005 # 提高（base：0.0001）
teacher_forcing_ratio = 0.3  # 降低（base：0.5）
clip = 5.0
n_epochs = 50000 # 考虑到基础模型的Loss曲线以及计算资源限制，不再增加迭代次数
```

4、调优结果比较

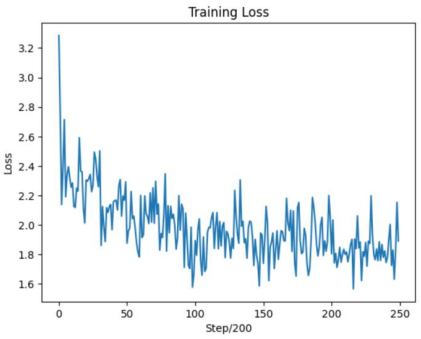
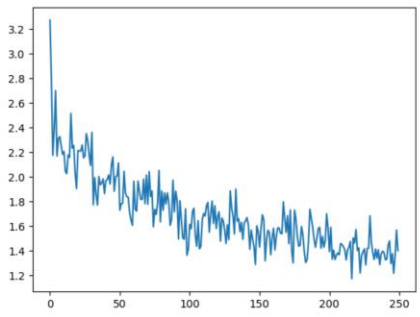
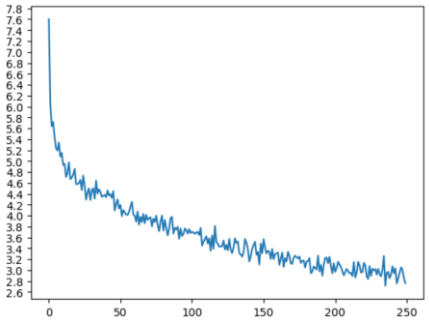
调优方法	Loss 曲线	参考翻译	BLEU
基础模型		====== 随机训练集 ====== > 我打算買支筆送給他。 = i ll buy a pen for him . < i m t to . . . . ====== 随机验证集 ====== > 他們在下棋。 = they re playing chess . < they re not . .	Corpus BLEU: 0.1519 Avarage Sentence BLEU: 0.1411
Beam Search	-	====== 随机训练集(beam search) ====== >我打算買支筆送給他。 i don t like to ====== 随机验证集(beam search) ====== >他們在下棋。 they don t have .	Corpus BLEU: 0.1263 Avarage Sentence BLEU: 0.1188
超参数		====== 随机训练集 ====== > 我打算買支筆送給他。 = i ll buy a pen for him . < i m to to you . . . . . ====== 随机验证集 ====== > 他們在下棋。 = they re playing chess . < he knows be to . . .	Corpus BLEU: 0.1174 Avarage Sentence BLEU: 0.1058

- 尝试的两种调优方法并不能提高 BLEU，反而 BLEU 降低了。
- 直接观察推理翻译，Beam Search 中加入了重复惩罚与长度惩罚，推理结果中确实没有了重复句号，句子长度也避免了过短，但除了句首代词，并没有新增有意义的单词。调高学习率后，loss 过早收敛，同样推理译文的质量没有提高。
- 在原 RNN 结构的基础上简单调优没有得到好的结果，考虑优化模型结构。

二、 模型结构优化

从简单到复杂尝试了两种结构的新模型：

- **GRU 模型**：用 GRU 代替编码器/解码器中的 RNN。
- **Attention 模型**：GRU 模型的基础上，编码器采用**双向 GRU**，解码器中加入 attention 机制。加入 attention 后参数会增加，模型能力增强，为了加快训练速度，hidden_size 设置为 256（基础模型和 GRU 模型为 500）。

	基础 RNN 模型	GRU 模型	Attention 模型
损失曲线			
推理样例	====== 随机训练集 ====== > 我打算買支筆送給他。 = i ll buy a pen for him . < i m t to . . . . ====== 随机验证集 ====== > 他們在下棋。 = they re playing chess . < they re not . .	====== 随机训练集 ====== > 我打算買支筆送給他。 = i ll buy a pen for him . < i ll call him to buy a . . ====== 随机验证集 ====== > 他們在下棋。 = they re playing chess . < they re here .	====== 随机训练集 ====== > 我们的花园布满了杂草。 = our garden was full of weeds . < our garden are full in the . ====== 随机验证集 ====== > 他的話讓我吃驚。 = his words surprised me . < his words me me .
BLEU	Corpus BLEU: 0.1519 Avarage Sentence BLEU: 0.1411	Corpus BLEU: 0.3231 Avarage Sentence BLEU: 0.3140	Corpus BLEU: 0.4123 Avarage Sentence BLEU: 0.3983

- 损失曲线平滑度：Attention > GRU > RNN。
- 从 Attention 模型的损失曲线明显观察到，迭代训练 50000 次还未收敛，需要更多的训练轮次。
- BLEU：Attention > GRU > RNN。
- 基础模型 BLEU 只有 0.15，几乎不能理解原文内容；而 GRU 和 Attention 模型能理解近一半的内容。

- 推理样例：GRU 和 Attention 模型的翻译准确性比基础模型有明显提高。首先，重复性句号大量减少；其次正确翻译比例明显增加，能正确翻译句子中一半左右的单词。从样例中无法明显看出 GRU 与 Attention 模型的优劣，但 Attention 的 BLEU 比 GRU 有约 27%的提高。

三、 批处理

从 Attention 模型的损失曲线明显观察到，迭代训练 50000 次还未收敛，需要更多的训练轮次。基础模型迭代训练的方法显然效率很低，无法充分利用 GPU。

修正模型代码实现批处理，尝试了两套超参数：

(A)

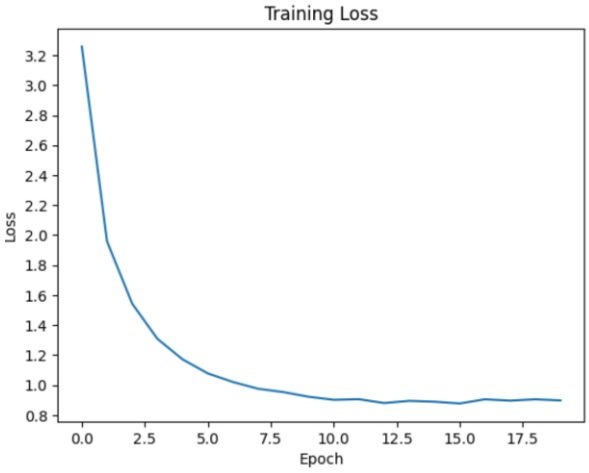
```
# ---------- 超参数 ----------
hidden_size = 256          # 隐藏层维度，可根据需要调整
n_layers = 2               # RNN层数
dropout = 0.1              # Dropout比例
learning_rate = 0.001
batch_size = 32
n_epochs = 20
clip = 5.0
teacher_forcing_ratio = 0.5
```

(B)

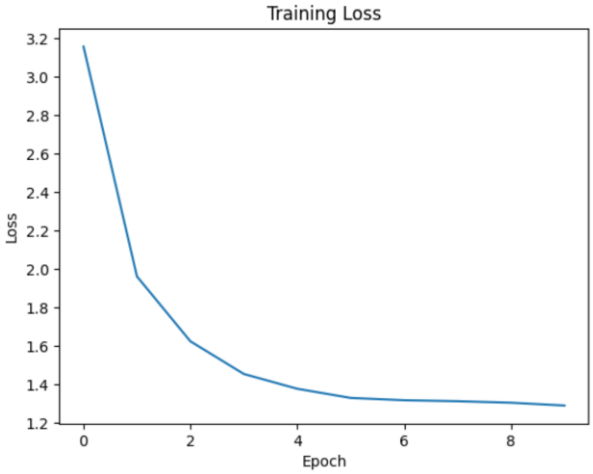
```
# ---------- 超参数 ----------
hidden_size = 512          # 增大(double)
n_layers = 2
dropout = 0.2              # 增大(double)
learning_rate = 0.001
batch_size = 32
n_epochs = 10
clip = 5.0
teacher_forcing_ratio = 0.5
```

训练损失曲线：批处理损失曲线平滑，避免模型陷入局部最优；loss 基本收敛。

(A)



(B)



推理结果（随机样例）：

(A)

```
====== 随机训练集 ======
batch_size=1
> 我已经不带眼镜了。
= i don t wear glasses anymore .
< i don t wear glasses anymore .


====== 随机验证集 ======
batch_size=1
> 你数学好吗?
= are you good in math ?
< are you good at mathematics ?
```

(B)

```
====== 随机训练集 ======
batch_size=1
> 你最喜欢的饮料是什么?
= what s your favorite drink ?
< what s your favorite ipad app ?


====== 随机验证集 ======
batch_size=1
> 没消息就是好消息。
= no news is good news .
< no news is good news .
```

- （A）在训练集上完全正确；在验证集上只有句末的介词加实体翻译错误。
- （B）在验证集上完全正确；在训练集上只有句末的实体翻译错误。

BLEU：

(A)

```
Corpus BLEU: 0.7496
Avarage Sentence BLEU: 0.7137
```

(B)

```
Corpus BLEU: 0.6840
Avarage Sentence BLEU: 0.6450
```

- 虽然采用了相同的网络结构，但批处理训练充分，比之前逐句训练的 Attention 模型（BLEU=0.4）有了大幅度提升。
- 选择 BLEU 值更大的（A）进行最终测试集推理翻译，并提交结果（*test_translations_batch.txt*）。

在（A）的基础上，还尝试了 Beam Search，比贪心算法的 BLEU 略有提高：

```
Corpus BLEU: 0.7684
Avarage Sentence BLEU: 0.7362
```